

Text : ที่มีความยาว n

output

search pattern

String Matching

1. Brute Force Algorithm (Sequential search)

Text	(a)	(a)	(a)	(a)	a	a	a	a	h
Pattern	(a)	(a)	(a)	(a)	(h)				
		(a)✓	(a)✓	(a)	(h)×				
			a	a	a	h			
				a	a		h		
					a	a		h	
						a	a	a	h

Text: aaaaaaaah
Pattern: aaah

∴ ทุกครั้งที่ mistake shift 1 ไป 1 ช่อง

Algorithm:

For(i=0 to n-m)

For (j=1 to m)

หากชุด
m วน

If(T[i+j] != P[j]) then break;
compare

End for

If(j>m) then print("match at i)

End for

Ex2.

Text: string Matching

pattern:

Area of improve !!

เพื่อใช้เร็วขึ้น

ลดจำนวนครั้งที่ทำการ compare

Complexity: O(mn)

2. Boyer-Moore Algorithm (matching อว → พิเศษ)

Bad-symbol shift

Bad-symbol shift → $d_1 = \max(1, t(c) - k)$, k is the number of matched characters

c	(a)	(b)	(c)	d	e	f	...	z
Shift t(c)	1	5	2	7	7	7	7	7

$m = \text{จำนวนตัวใน pattern}$

Text	a	b	a	b	a	b	(a)	c	b	d	c	a	c	a	c
Pattern	a	b	a	a	c	a	(a)	c	b	d	c	a	c	a	c

$$d_1 = \max(1, 1 - 0) = 1$$

Text	a	b	a	b	a	(b)	(a)	(c)	b	d	c	a	c	a	c
Pattern	a	b	a	a	a	(b)	(a)	(c)	b	d	c	a	c	a	c

shift 1 point

$$d_1 = \max(1, 5 - 2) = 3$$

Text	a	b	a	b	a	b	a	c	b	(d)	(c)	a	c	a	c
Pattern	a	b	a	a	b	a	a	a	c	(d)	(c)	a	c	a	c

shift 3 point

$$d_1 = \max(1, 7 - 1) = 6$$

Text	a	b	a	b	a	b	a	c	b	d	c	a	(c)	(a)	(c)
Pattern	a	b	a	a	b	a	a	a	c	b	d	a	(a)	(c)	(c)

shift 6 point

Good-suffix shift

Good-suffix shift

k	pattern	d_2
1	a b a a (c) a c	2
2	a b a (a c) a c	2

matched char

pre process

Text	a	b	a	b	a	(b)	(a)	(c)	b	d	c	a	c	a	c
Pattern	a	b	a	a	a	(b)	(a)	(c)	b	d	c	a	c	a	c

$$d_2 = 2$$

Text	a	b	a	b	a	b	a	c	b	d	c	a	c	a	c
Pattern	a	b	a	a	b	a	a	c	a	c					

shift 2 point

If $k = 0$ then $d = d_1$

Else $d = \max(d_1, d_2)$

ถ้าไม่มี

Example 1

c	A	B	C	D	...	O	...	Z	—
Shift t(c)	1	2	6	6	6	3	6	6	6

$$d_1 = \max(1, t(c) - k)$$

k	pattern	d_2
1	BAO <u>B</u> A <u>B</u>	2
2	<u>B</u> AOBAB	5
3	<u>B</u> AO <u>B</u> A <u>B</u>	5
4	<u>B</u> A <u>O</u> B <u>A</u> B	5
5	<u>B</u> A <u>O</u> B <u>A</u> B	5

Text B E S S _ **K** N E W _ A B O U T _ B A O B A B S
Pattern B A O B A **B**
 $k=0 \rightarrow d=d_1 = \max(1, 6-0) = 6$ $f(k)=6$

Text B E S S _ K N E W B A B O U T _ B A O B A B S
Pattern B A O B A B

k=2 → d=max(d1,d2) = max(6-2,5) = 5 +(-)=6 $d_1=4$
 $d_2=d(2)=\frac{5}{6 \times 5}$

Text B E S S _ K N E W _ A B O U T **A** B A O B A B S
Pattern B A O B A B S
 $k=1 \rightarrow d=\max(d_1,d_2) = \max(6-1,2) = 5$

Text B E S S _ K N E W _ A B O U T _ B A O B A B S
Pattern k = 6 = m → four Match

จำนวนการเปรียบเทียบ = 12 ครั้ง (ถ้าใช้ **brute force** ต้องเปรียบเทียบกี่ครั้ง?)

Complexity: Worst case ใช้เวลา nm ; Best case ใช้เวลา n/m

Note: Boyer –Moore's Algorithm is significantly faster than brute-force algorithm on English text

Example 2

[illegible]

k	pattern	d ₂
1	GTAGCGGCG	2
2	GTAGCGGCG	3
3	GTAGCGGCG	3
4	GTAGCGGCG	8

Text G T T A T A G C **T** G A T C G C G G C G T A G C G G C G
Pattern G T A G C G G C **G**
 $k=0, t(T)=7, \therefore d_1 = \max(1, 7-0) = 7, d_2 = \infty$

Text G T T A T A G C T G A T **C** **G** **C** **G** G C G T A G C G G C G
Pattern G T A G C **G** **G** **C** **G**

$k=3, +(T)=1, d_1=\max(1, 1-3)=1; d_2=3$

Text G T T A T A G C T G A T C G C G G C G T A G C G G C G
 Pattern G T A G C G G C G
 $k=6; t(T)$

Text G T T A T A G C T G A T C G C G G C G T A G C G G C G

Pattern G T A G C G G C G

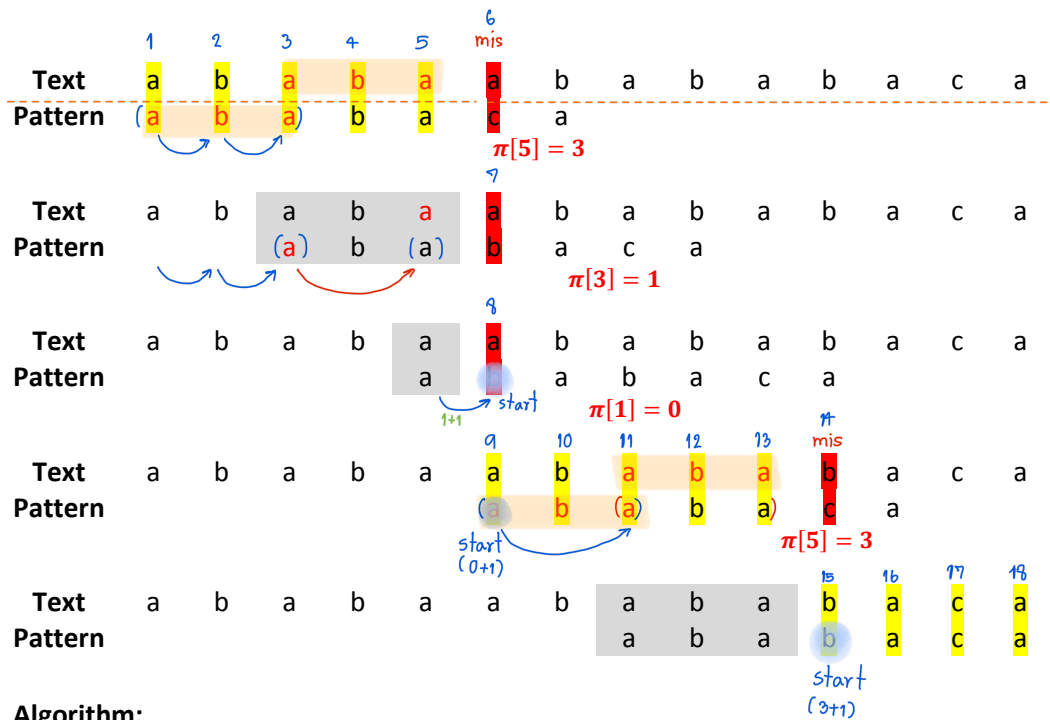
3. Knuth-Morris-Pratt's Algorithm : KMA : left to right

Text	a	b	a	b	a	a	b	a	b	a	b	a	c	a			
Pattern	a	b	a	b	a	c	a										

↑ mismatch

preprocess :

k	1	2	3	4	5	6	7
π	0	0	1	2	3	0	1



Algorithm:

$\pi = \text{compute-Prefix-Function}(P)$ // ใช้เวลา m

q=0

for(i=1 to n){

while(q>0 and P[q+1] $\neq$ T[i]){

q = $\pi[q]$

}

If (P[q+1] $\neq$ T[i]) q ++

If (q==m){

Print(i-m)

q = $\pi[q]$

}

}

index i
move

Time Complexity: n + m

Text	a	b	a	b	a	a	b	a	b	a	b	a	c	a
Pattern	a	b	a	b	a	c	a							

compute-Prefix-Function(P)

q=1; $\pi[1] = 0$ ไม่ซ้ำ

a b a b a c a

q=2 $\pi[2] = 0$ ไม่ซ้ำ prefix, suffix

a b a b a c a
a b a b a c a

q=3 $\pi[3] = 1$

a b a b a c a
a b a b a c a

aba
(a)ba

q=4 $\pi[4] = 2$

a b a b a c a
a b a b a c a

ab(ab)
(a b)ba

q=5 $\pi[5] = 3$

a b a b a c a
a b a b a c a

ab(ab)a
(aba)ba ca

q=6 $\pi[6] = 0$ ไม่ตรงเลย

a b a b a c a
a b a b a c a

ababac a
ababac a

q=7 $\pi[7] = 1$

a b a b a c a
a b a b a c a

ababac a
ababac a

Algorithm:

```

compute-Prefix-Function(P[1..m]){
     $\pi[1] = 0; k = 0$ 
    for(q = 2 to m){
        while(k > 0 and P[k+1] != P[q]){
             $k = \pi[k]$ 
        }
        If (P[k+1] == P[q]) k++
         $\pi[q] = k$ 
    }
    return  $\pi$ 
}

```

Pattern : AGACAG

AGACAG
AGACAG

คำนวณหา π

k 1 2 3 4 5 6

π 0 0 1 0 1 2

Text AGACAAGACCAG ACAG

✓✓✓✓✓
AGACAG $k=5$ $\pi(5)=1$

^x
(A)GACAG $k=1$, $\pi(1)=0$

✓✓✓✓
AGACAG $k=4$, $\pi(4)=0$

✓✓✓✓✓
AGACAG